hello gpt, how are you? i need you to act as a personal trainner for Dh bike racers and build me a complete guide for trainning on preseason

Hey! I'm doing great, thanks! 🤘 And absolutely — let's approach this like a professional coaching program for **Downhill (DH) mountain bike racers**.

The goal of preseason isn't just to get stronger in the gym. It's to build a rider who can handle repeated high-intensity efforts, stay composed through rough terrain, maintain grip and control when fatigued, and arrive at race season strong, explosive, and injury-resilient.

I'll lay this out as a practical coaching guide covering physical preparation, gym work, bike skills, conditioning, recovery, and progression. The plan will assume you're preparing for gravity racing, where strength, power, technical precision, and the ability to repeat hard runs matter more than long-distance endurance alone.

DH mountain bike racing: complete preseason training guide



1. The objectives of preseason

A good preseason usually lasts **8–12 weeks**, depending on your current fitness and how far away the first race is.

Maximum strength	

Develop strong legs, hips, back, shoulders, and grip so you can manage impacts, braking forces, and bike movement.

Explosive power

Improve how quickly you can produce force, accelerate out of corners, pump terrain, and respond to changing trail features.

Technical riding and repeatability

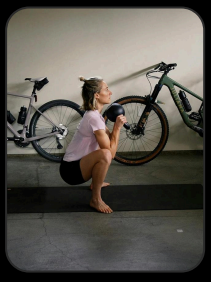
Practice braking, cornering, line choice, body position, and repeated high-quality runs without allowing fatigue to destroy technique.

Durability and recovery

Build trunk stability, mobility, aerobic fitness, and enough recovery capacity to handle multiple runs and training days.

2. The 12-week preseason structure

The plan is divided into four phases. These are starting guidelines, not fixed deadlines; progress depends on training experience, age, recovery, and riding access.



Weeks 1–3

Phase 1 — General preparation

Build the foundation

- Learn correct lifting technique.
- Improve mobility and movement quality.
- Develop basic aerobic fitness.
- Begin progressive core and grip training.

Intensity: moderate



Weeks 4–6

Phase 2 — Strength development

Build force production

- Progress to heavier compound lifts.
- Increase unilateral leg strength.
- Train upper-body pulling and pushing.
- Introduce low-volume explosive exercises.

Intensity: moderate to high



Weeks 7–9

Phase 3 — Power and race conditioning

Convert strength into performance

- Combine strength and explosive work.
- Add short, hard intervals.
- Increase technical riding intensity.
- Practice repeated runs with controlled recovery.

Intensity: high, with recovery days



Weeks 10–12

Phase 4 — Race preparation

Become fast, fresh, and consistent

- Prioritize race-like runs and line selection.
- Maintain strength with less gym volume.
- Practice starts and full-course pacing.
- Reduce fatigue before the first event.

Intensity: race-specific; volume decreases

3. Weekly training schedule

This is a sample week for a healthy, intermediate adult rider who already tolerates regular training. It assumes access to a gym and at least some trail riding.

Use this as the basic structure throughout preseason, adjusting the workload to the current phase.

Day	Main session	Duration
Monday	Lower-body strength + core	60–75 min
Tuesday	Bike skills + aerobic conditioning	60–90 min
Wednesday	Upper-body strength + grip	50–70 min
Thursday	Recovery, mobility, easy spin	30–45 min
Friday	Full-body power + short intervals	45–65 min
Saturday	DH riding, technique, repeated runs	90–180 min
Sunday	Full rest or easy recovery	0–30 min

Important: This is a starting template, not seven mandatory training days. If you are new to structured training, start with three or four training days and build up. During demanding riding weekends, remove or reduce Friday's hard conditioning session.

4. Gym training: the core exercises

Session A — Lower-body strength



1. Squat variation

3–5 sets × 4–8 reps

Back squat, front squat, or goblet squat depending on experience.

Develops leg and hip strength for supporting the rider through compressions and rough terrain.



2. Romanian deadlift

3–4 sets × 6–8 reps

Strengthens the hamstrings, glutes, and posterior chain, helping with hip control and maintaining a strong riding position.



3. Bulgarian split squat

3 sets × 6–8 per leg

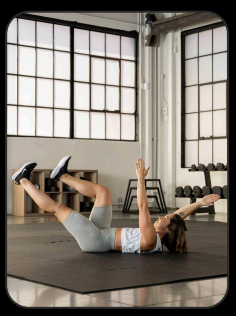
Improves single-leg strength, balance, and control through uneven loading.



4. Calf raises

3 sets × 10–15 reps

Supports ankle strength and lower-leg capacity.



5. Dead bug

3 sets × 8–12 per side

Builds trunk control while the arms and legs move independently.

Rest 2–3 minutes between demanding compound sets and approximately 60–90 seconds for accessory exercises. Keep 2–3 good repetitions in reserve during the first few weeks. Never sacrifice technique to lift heavier.

Session B — Upper body, trunk, and grip



Pull-ups or lat pulldowns

3 × 6–10 reps

Build pulling strength for bike control.



Dumbbell bench press or push-ups

3 × 6–12 reps

Develop chest, shoulder, and triceps strength.



Single-arm dumbbell row

3 × 8–12 per side

Supports upper-back strength and shoulder control.



Pallof press

3 × 10 per side

Trains anti-rotation and trunk stability.



Farmer carries

3 × 30–45 seconds

Develop grip, forearm, shoulder, and trunk endurance.

For DH racers, grip training should be progressive. Excessive forearm fatigue can interfere with braking control and increase risk during riding. Avoid exhausting your grip before a technical trail session.

Session C — Power and conditioning

Start this session only after building a sound strength base and learning to land and move explosively with good control.

Exercise	Prescription	Focus
Box jumps	3 × 3–5	Explosive lower-body power
Medicine-ball slams	3 × 5–8	Whole-body power
Kettlebell swings, if skilled	3 × 8	Hip power
Push-ups	2–3 × 8–12	Upper-body strength

Side plank	3 × 30–45 sec/side	Lateral trunk stability

Rest adequately between explosive sets. Every repetition should be fast and technically clean. Stop when jump height, landing quality, or speed visibly declines.

During weeks 1–3, prioritize movement quality rather than maximizing load or explosive output. In weeks 4–6, build strength. In weeks 7–9, maintain strength while emphasizing quality power work. In weeks 10–12, reduce gym volume and preserve freshness.

5. Bike-specific conditioning

A DH run may be short, but it can involve repeated hard accelerations, braking, gripping, and full-body stabilization. You need both an aerobic foundation and high-intensity capacity.



A. Aerobic base

1–2 sessions per week, 30–60 minutes at an easy conversational pace.

Use a bike, easy trail ride, or another low-impact activity. The purpose is to improve recovery between hard efforts, not to exhaust yourself.



B. Short high-intensity intervals

Begin with 4–6 × 20–30 seconds hard, with 2–3 minutes of easy recovery.

Progress gradually toward 6–8 repetitions if you recover well. Use a safe, predictable climb, stationary bike, or controlled training area.



C. DH-specific repeated runs

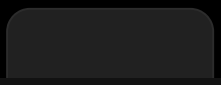
Begin with 3–4 quality runs and adequate recovery. Gradually increase the number of runs according to trail difficulty and how well technique holds up.

Prioritize line choice, braking points, corner exits, and consistency over simply accumulating fatigue.

Do not perform maximum-intensity intervals every day. One demanding conditioning session per week is a sensible starting point, with an additional session only when your riding and gym workload allow it.

6. Technical riding: what to train

Physical fitness is only one part of DH performance. Use preseason to deliberately practice individual skills.



Braking control



Practice progressive braking, choosing braking points, and releasing the brakes before corners where appropriate. Develop consistency rather than relying on panic braking.



Cornering

Work on vision, entry speed, body-bike separation, traction, and smooth exits. Repeat the same corner to identify improvements.



Rough terrain and body position

Practice staying balanced, absorbing terrain with arms and legs, and maintaining a neutral athletic stance. Progress from easier features to more demanding terrain.



Line choice and vision

Look ahead, identify braking points, and select repeatable lines. When possible, review video to identify where speed is lost or technique deteriorates.

Track a few useful measures: run time on the same course, consistency across runs, perceived effort, and the number of avoidable mistakes. A faster run is not necessarily a better training run if it comes with significantly more risk.

7. Mobility and injury resilience

Spend 8–12 minutes on mobility most training days, focusing on areas that limit your riding position.

- **Ankles:** knee-to-wall ankle mobilization, 2 × 8 per side.
- **Hips:** controlled 90/90 rotations, 2 × 8 per side.
- **Thoracic spine:** open-book rotations, 2 × 8 per side.
- **Shoulders:** controlled arm raises or wall slides, 2 × 10.
- **Wrists and forearms:** gentle wrist movements and progressive forearm strengthening.

Add targeted work for any known limitations with an appropriate physiotherapist or qualified coach. Strength and mobility training can improve physical capacity, but they cannot eliminate injury risk.

8. Nutrition and recovery

Your training only works if you recover from it.

Protein

Aim for approximately 1.4–2.0 g/kg of body weight per day, distributed over 3–5 meals, to support muscle repair and adaptation.

Carbohydrates

Eat enough to support demanding gym sessions, intervals, and long riding days. Include carbohydrate-rich foods before and after hard sessions.

Hydration

Drink regularly and adjust fluid and electrolyte intake to temperature, duration, and your sweat losses, particularly during hot riding days.

Sleep

Aim for 7–9 hours per night as an adult; adolescents generally need more. Keep sleep regular and add recovery time when training load increases.

You do not need an aggressive diet or unnecessary supplements to get faster. Maintain sufficient energy intake, especially during a demanding preseason. If you're still growing, avoid weight-loss-focused programming without appropriate professional guidance.

9. How to monitor progress

Use a simple training log after each session.

Daily athlete check-in

Complete this before deciding whether to push hard or reduce the day's workload.

Sleep last night

- ○ Under 6 hours
- ○ 6–7 hours
- ◉ 7–9 hours
- ○ Over 9 hours

Energy today

- ○ Low
- ○ Normal

Good

Muscle soreness

None

Mild

Moderate

Severe or unusual pain

Session status

Completed as planned

Reduced the workload

Rested or skipped

Notes (optional)

e.g. forearms fatigued early, legs felt strong, trail was wet...

Copy daily log

Review the log weekly. If performance is falling, soreness persists, sleep deteriorates, or motivation drops for several days, reduce the training load rather than automatically adding more work.

10. What should change as race season approaches?

Training element	Early preseason	Mid-preseason	Race preparation
Gym strength	Build foundation	Heavier, controlled work	Maintain with lower volume
Explosive power	Learn technique	Increase gradually	Low volume, high quality
Aerobic conditioning	Build base	Maintain and develop	Maintain
High-intensity work	Limited	Gradually increase	Race-specific
DH riding	Technique basics	More demanding trails	Race runs and starts
Total training volume	Moderate	Highest tolerable workload	Reduce to arrive fresh

In the final 7–10 days before a key race, reduce total training volume while keeping a small amount of familiar, high-quality intensity. Avoid new exercises, unusually heavy lifting, or exhausting interval sessions close to race day.

Let's build your personal DH racer program

The guide above is the general framework. To coach you properly, I need to tailor it to your experience, schedule, equipment, and race calendar.

Athlete profile

Choose what best describes you. You can leave the defaults or tell me more in the final field.

1. Age range

- ○ Under 18
- ◉ 18–29
- ○ 30–39
- ○ 40+

---

2. Current DH level

- ○ Beginner — learning the basics
- ◉ Intermediate — regular trail rider
- ○ Advanced — experienced racer
- ○ Elite / expert racer

3. When is your first important race?

- ○ In 4–6 weeks
- ○ In about 8 weeks
- ◉ In about 12 weeks
- ○ More than 12 weeks away
- ○ No race scheduled yet

---

4. Gym equipment

- ◉ Full gym — barbells, dumbbells, machines

- ○ Basic home gym
- ○ Bodyweight only

5. How often can you ride DH?

- ○ 1 day per week
- ◉ 2 days per week
- ○ 3 or more days per week
- ○ Limited access to DH trails

6. Total training days available per week

- ○ 3 days
- ○ 4 days
- ◉ 5 days
- ○ 6 days

7. Injuries, limitations, or other details

None

Build my personalized DH preseason plan

Once we have your profile, we can turn this into a structured day-by-day coaching plan with exact exercises, sets, repetitions, rest periods, bike workouts, and weekly progression.

I can also create a visual preseason training calendar you can keep as a reference in your gym or bike workshop.

i love it!, but i want to build an app that can be as generic as posible, but at the same time, tailored for each individual profile, as you were triying to ask me

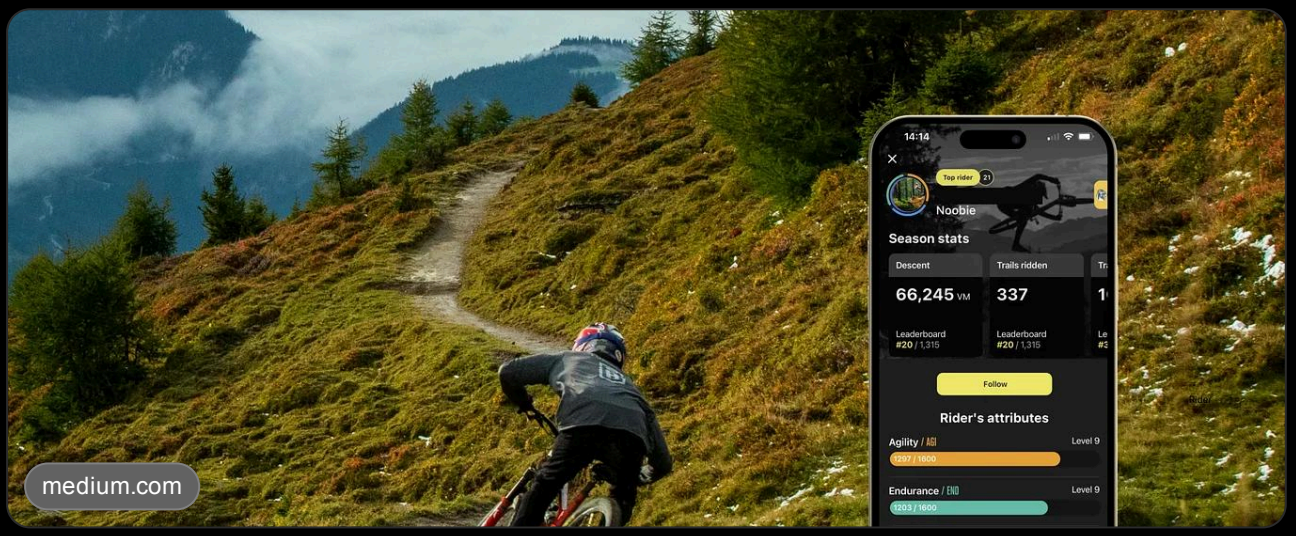
I love this direction. What you're describing is much more than a training-plan generator: it's a **personalized coaching platform for downhill mountain bike racers**, built around one flexible system that adapts to each athlete.

The key product principle should be:

One coaching engine. One shared training methodology. A unique plan for every rider.

You don't want to build a separate program for beginners, elite racers, riders without a gym, and athletes with only three days available. You want an app that understands those differences and adjusts the program automatically.

1. The app concept



DH Coach

Working name · Personalized performance coaching for gravity athletes

The app should help a rider answer five questions:

- What should I train today?
- Why am I doing this session?
- How should the workout adapt to my level and equipment?
- Am I getting fitter, stronger, and faster?
- What should change as my next race approaches?

The user experience should feel like having a personal coach, not filling out a spreadsheet.

2. The product architecture

I would build the app around four connected layers.

1. Athlete profile

Experience, goals, age, availability, equipment, injuries and preferences

↓

2. Coaching engine

Rules, training phases, exercise library, workload limits and progression

↓

3. Personalized plan

Daily sessions, gym workouts, bike skills, conditioning and recovery

↓

4. Feedback and adaptation

Workout completion, fatigue, progress and automatic plan adjustments

This separation matters. The coaching engine defines what makes a good training program. The personalization layer decides which version is appropriate for a particular athlete. The feedback layer adjusts it over time.

AI can help explain sessions, answer coaching questions, and suggest substitutions, while explicit rules govern progression, safety limits, and training load.

3. Design the onboarding system

This is the most important part of the app. Instead of asking every possible question at once, use a **progressive athlete assessment**: ask essential questions first, then request more detail when it improves the plan.



Step 1 — Who is the athlete?

- Age or age bracket
- Riding discipline: DH, enduro, freeride, or mixed
- Racing experience and competition category
- Main goal: fitness, speed, consistency, or first race

Purpose: establish appropriate training and safety boundaries.

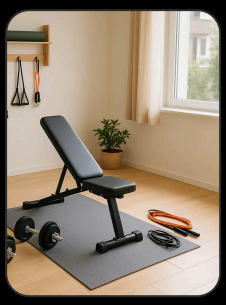
Step 2 — What is their current level?

- Riding frequency and recent training history
- Gym experience and current strength-training routine
- Technical confidence and access to trails



- Current conditioning and recent performance

Purpose: establish the starting workload instead of assuming every athlete is intermediate.



Step 3 — What resources are available?

- Gym, home gym, or bodyweight only
- DH trails, bike park, pump track, or indoor trainer
- Training days and available time per session
- Coaching, training partners, and equipment constraints

Purpose: produce a plan the athlete can actually follow.



Step 4 — What are they preparing for?

- First race date or target event
- Number of races and priority events
- Current training phase
- Time available before competition

Purpose: work backward from the race calendar to create appropriate training blocks.



Step 5 — How is the athlete recovering?

- Sleep and perceived energy
- Training soreness and fatigue
- Previous injuries or current limitations
- Work, school, and other life demands

Purpose: determine whether to progress, maintain, modify, or recommend rest.

The onboarding rule

Don't force users to complete a 30-question form before seeing value.

Start with perhaps 8–10 essential questions. Generate a provisional plan, then ask optional questions that improve exercise selection, training intensity, and adaptation. Never require an athlete to disclose unnecessary medical information; collect only what is needed to support safe training.

4. Build a flexible athlete profile

Every answer should influence one or more specific parts of the program.

Profile variable	What the app changes

Age and development stage	Training suitability, supervision, and recovery
Experience level	Exercise complexity, volume, and progression
Days available	Weekly schedule and session distribution
Gym equipment	Exercise selection and alternatives
Riding access	Trail sessions versus alternative conditioning
Race date	Training phase and taper timing
Current fitness	Starting workload and conditioning
Recovery feedback	Daily intensity and workload adjustments
Goals	Relative emphasis on strength, power, skills, or endurance

A crucial design distinction: **training level is not the same as racing level**. An expert rider may be inexperienced in the gym. A beginner racer may have years of strength-training experience. The app should model these separately.

Similarly, a previous injury should not automatically trigger a diagnosis or rehabilitation program. It should inform exercise constraints and, where appropriate, prompt the athlete to seek professional guidance.

5. The coaching engine: how personalization actually works

Avoid relying on a single AI prompt to invent a complete training plan from scratch every time. That can lead to inconsistent workouts, unrealistic workloads, and unsafe progression.

Instead, use a structured exercise library and training rules.

Example: one goal, three personalized workouts

All three target lower-body strength, but the session changes according to the athlete's experience and resources.

Athlete A — Beginner Home

35–45 minutes

- Goblet squat or chair squat
- Hip bridge
- Supported split squat
- Dead bug

Focus: learn movement patterns and establish consistency.

Athlete B — Intermediate Full gym

50–65 minutes

- Squat variation
- Romanian deadlift
- Bulgarian split squat
- Calf raises and trunk stability

Focus: progressive strength development.

Athlete C — Advanced Full gym

55–70 minutes

- Individualized heavy compound lift
- Posterior-chain strength
- Unilateral strength
- Carefully dosed explosive work

Focus: maximize useful force while managing fatigue and preserving riding quality.

The app should choose a workout using a sequence like this:

1. Identify the athlete's training phase and target adaptation.
2. Filter exercises by equipment, experience, and stated limitations.
3. Select an appropriate starting volume and intensity.
4. Fit the session into the athlete's available time.
5. Check that the session is compatible with the rest of the week.
6. Present the workout with instructions, alternatives, and a clear purpose.
7. Use completion and recovery feedback to guide the next session.

Each exercise should have structured metadata: movement pattern, muscle groups, equipment, difficulty, coaching cues, contraindication flags, progression, regression, sets, reps, rest, and intended training effect.

This library becomes one of your app's most valuable assets.

6. The app should adapt every week

A genuinely personalized coach doesn't just generate a plan once. It learns from how the athlete responds.

For example:

Adaptive coaching example

The athlete completes all prescribed sets

Technique is solid, recovery is good, and the session feels appropriate. The app may progress repetitions or load at the next planned opportunity.

The athlete feels unusually fatigued

The app reduces or replaces demanding work and checks whether the fatigue persists. It doesn't automatically compensate by adding more work later.

The athlete misses a training day

The app intelligently reschedules the priority sessions without cramming missed workouts into the remaining days.

Race day approaches

The app adjusts the training phase, reduces accumulated fatigue, and preserves short, familiar efforts that help the athlete feel sharp.

For the first version, use simple, transparent rules. Later, you can add wearable data, performance trends, automated load recommendations, and more sophisticated AI coaching.

7. What I would build for the MVP

Don't start by building every possible feature. Start with a product that can reliably generate and manage a personalized preseason plan.

Feature 1 — Athlete onboarding

Profile, training history, goals, equipment, schedule, and race dates.

Feature 2 — Personalized training calendar

Weekly schedules with gym, riding, conditioning, mobility, and recovery sessions.

Feature 3 — Workout details

Sets, reps, rest, exercise demonstrations, coaching cues, and equipment-based alternatives.

Feature 4 — Training log

Session completion, perceived effort, fatigue, soreness, and progress.

Feature 5 — AI coach

Explain the plan, answer training questions, suggest approved substitutions, and help athletes understand their progress.

8. A practical technical architecture

For a web-first MVP, I'd consider this setup:

Component	Suggested approach
User interface	Responsive web app; Next.js or a comparable framework
Database and authentication	Supabase with PostgreSQL
Training-plan engine	Structured rules and templates in application code
AI coaching	An LLM API with constrained, structured inputs and outputs
Exercise library	Database-backed exercise catalog with demonstrations
Analytics	Training history, adherence, workload, and progress metrics

Keep the athlete profile, plan version, workout prescriptions, completed sessions, and feedback as separate records. That makes it possible to regenerate future sessions without losing what was originally prescribed.

Personal data should be protected with appropriate access controls. AI-generated recommendations should pass through validation rules before being saved or shown to the athlete.

9. Let's define what we're actually building

There are two decisions that will shape the entire project:

- **Product scope:** are we building a DH-specific coaching product first, with room to expand into enduro and other disciplines, or a multi-discipline platform from day one?
- **Build approach:** are you looking for a working prototype you can test immediately, or the architecture and implementation plan for a production-ready app?

Define your first version

Which riders will it serve?

- ○ Downhill racers only
- ○ DH first, expandable to enduro and other disciplines
- ◉ DH, enduro, XC, and other disciplines from launch

What do you want to build first?

- ○ A working interactive prototype I can test
- ◉ A production app architecture and implementation plan
- ○ A no-code MVP I can launch quickly

Who is the intended customer?

- ○ Individual riders training independently
- ○ Coaches managing multiple athletes
- ◉ Both individual riders and coaches

Where should people use it?

- ○ Web app, usable on desktop and mobile
- ○ Native mobile app
- ◉ Both web and mobile apps

Define my app and next steps

My recommendation is to start with **DH first, individual riders, and a working web prototype**. Validate that the onboarding produces sensible plans for very different athletes before expanding the product. That will give you a solid coaching system to build on rather than a beautiful app that generates generic workouts.

Help me build the DH athlete coaching platform. Product scope: Multiple disciplines from day one. Initial build approach: Production app architecture. Target customers: Both individual riders and coaches. Platform: Web app and mobile app. Based on these choices, define the MVP, architecture, data model, onboarding flow, personalized training engine, and a concrete implementation roadmap.

Gravity Coach — product and engineering blueprint

You're building a multi-discipline training platform for gravity and mountain bike athletes, serving both individual riders and professional coaches, with web and mobile experiences.

My recommendation is to build this as a **multi-tenant coaching platform with a shared training engine, configurable discipline-specific rules, and individualized training**

plans. That architecture will let you support downhill, enduro, cross-country, and other disciplines without building a separate app for each one.

The most important architectural decision is to separate the *training methodology* from the *athlete's individual plan*. Your methodology should be reusable, testable, and versioned. Personalization should determine which parts of that methodology apply to each athlete.

I'll use **Gravity Coach** as a working name; it can be changed later.

1. Product vision and core users



Individual athletes

Receive personalized training plans, follow daily workouts, record results, and adapt training around their availability, equipment, recovery, and race calendar.



Coaches and teams

Manage multiple athletes, create reusable training templates, assign and adjust plans, review training logs, and monitor athlete progress.



Platform administrators

Maintain the exercise library, coaching methodology, discipline configurations, content, permissions, and platform-wide safety policies.

Product principles

1. One account can participate in multiple disciplines.
2. An athlete can belong to a coach or team without losing ownership of their own training history.
3. Coaches can override automated recommendations.
4. Plans must work with different equipment, schedules, experience levels, and training environments.
5. The AI explains and assists the coaching process; deterministic rules govern the essential safety and workload constraints.
6. Every prescribed workout should have a clear reason for being included.

2. Define the MVP

The MVP should prove that the platform can create, deliver, and adapt a credible training plan for different types of athletes, and that coaches can manage those athletes efficiently.

P0 — Required for launch

- Registration, login, account recovery, and profiles
- Athlete onboarding and training availability
- Multiple disciplines and training goals
- Configurable training phases and race calendars
- Exercise library with equipment requirements and instructions
- Plan generation and a weekly calendar
- Workout completion and training logs
- Recovery check-ins and basic plan adjustments
- Coach workspace, athlete invitations, and plan review
- Role-based access control and audit history

P1 — Shortly after launch

- Athlete comparison and progress dashboards for coaches
- Reusable workout and training-block templates
- Coach comments and athlete messaging
- Push notifications and reminders
- Plan version history and change explanations
- Structured AI coaching assistant
- Exportable training reports

P2 — Expansion

- Wearable and activity-platform integrations
- Advanced performance analytics
- Training-load and readiness models validated against real athletes
- Team subscriptions and advanced coach workflows
- Event calendars, race analytics, and more discipline-specific modules
- Offline-first mobile training logs

Keep out of the initial MVP: automatic diagnosis or rehabilitation, black-box training prescriptions, complex biometric predictions, and a large social network. Those features increase cost and risk before the core product has been validated.

A reasonable initial validation target is a small pilot with a handful of coaches and several dozen athletes, covering beginners through advanced riders and at least three distinct disciplines. Treat that as a testing target, not a forecast of commercial success.

3. Production architecture

I recommend a **modular monolith first, with clear boundaries between modules**. This is easier to develop, deploy, test, and maintain than starting with microservices, while leaving room to split out high-volume services later.

Recommended technology stack

Web and mobile clients

Next.js + React Native / Expo + TypeScript

The web app supports athletes and coach dashboards. Expo provides iOS and Android apps using shared TypeScript models and reusable business logic.

Backend and API

NestJS or a comparable TypeScript API

Owns authorization, plan generation, workout logging, coach workflows, validation, and domain rules. Use a versioned REST API with OpenAPI documentation.

Database and authentication

PostgreSQL + managed authentication

Supabase is a practical starting point for PostgreSQL, authentication, and file storage. The backend remains the authority for business logic and privileged operations.

Background processing

Managed queue + workers

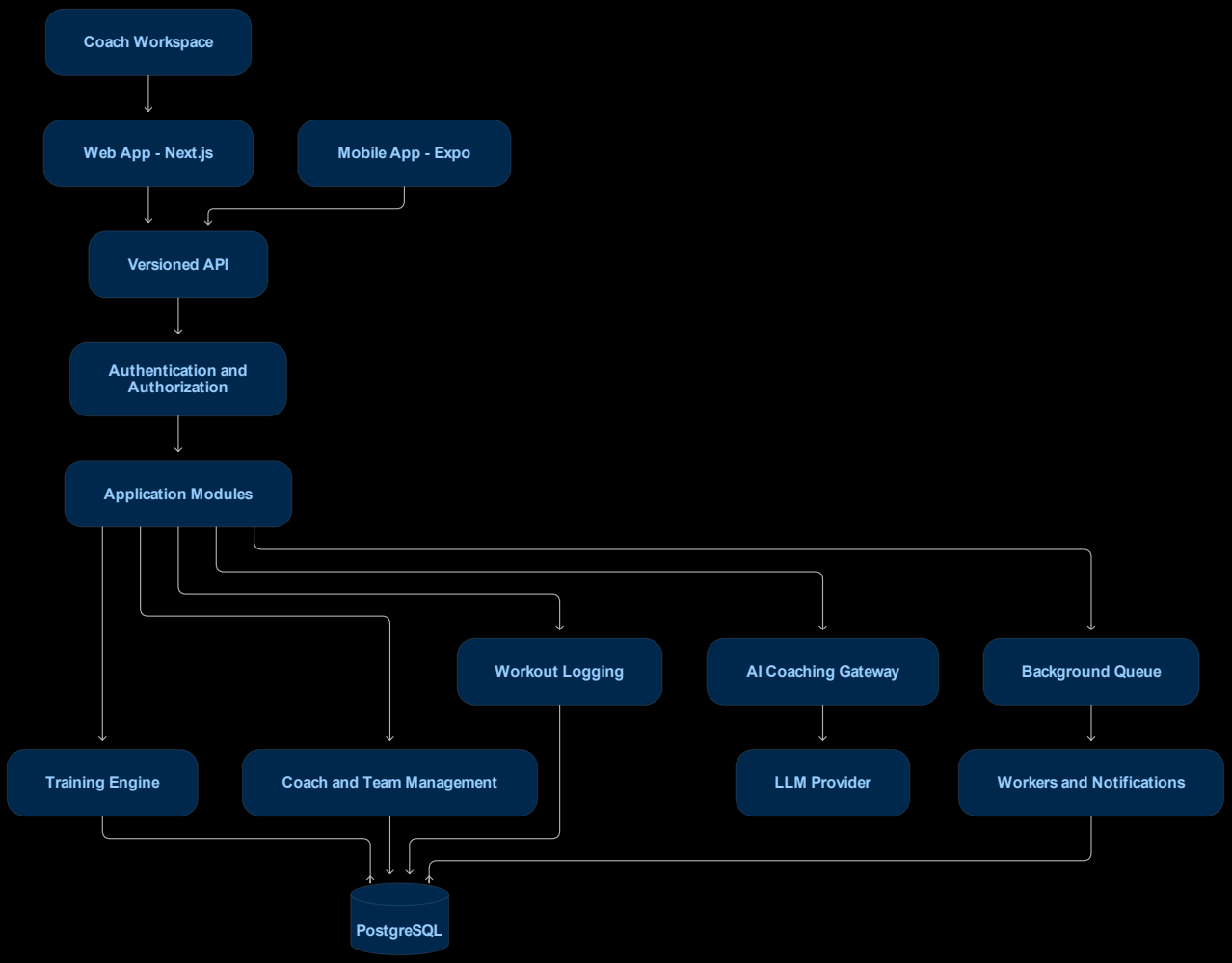
Handle plan generation, reminders, report generation, integrations, and retryable jobs outside the request-response cycle.

AI and coaching intelligence

LLM API + deterministic training engine

The rules engine generates valid prescriptions. AI explains workouts, answers questions using approved coaching content, and proposes changes for validation.

System diagram



Recommended codebase structure

Use a monorepo so your web and mobile clients can share domain types and validation schemas.

Plain text

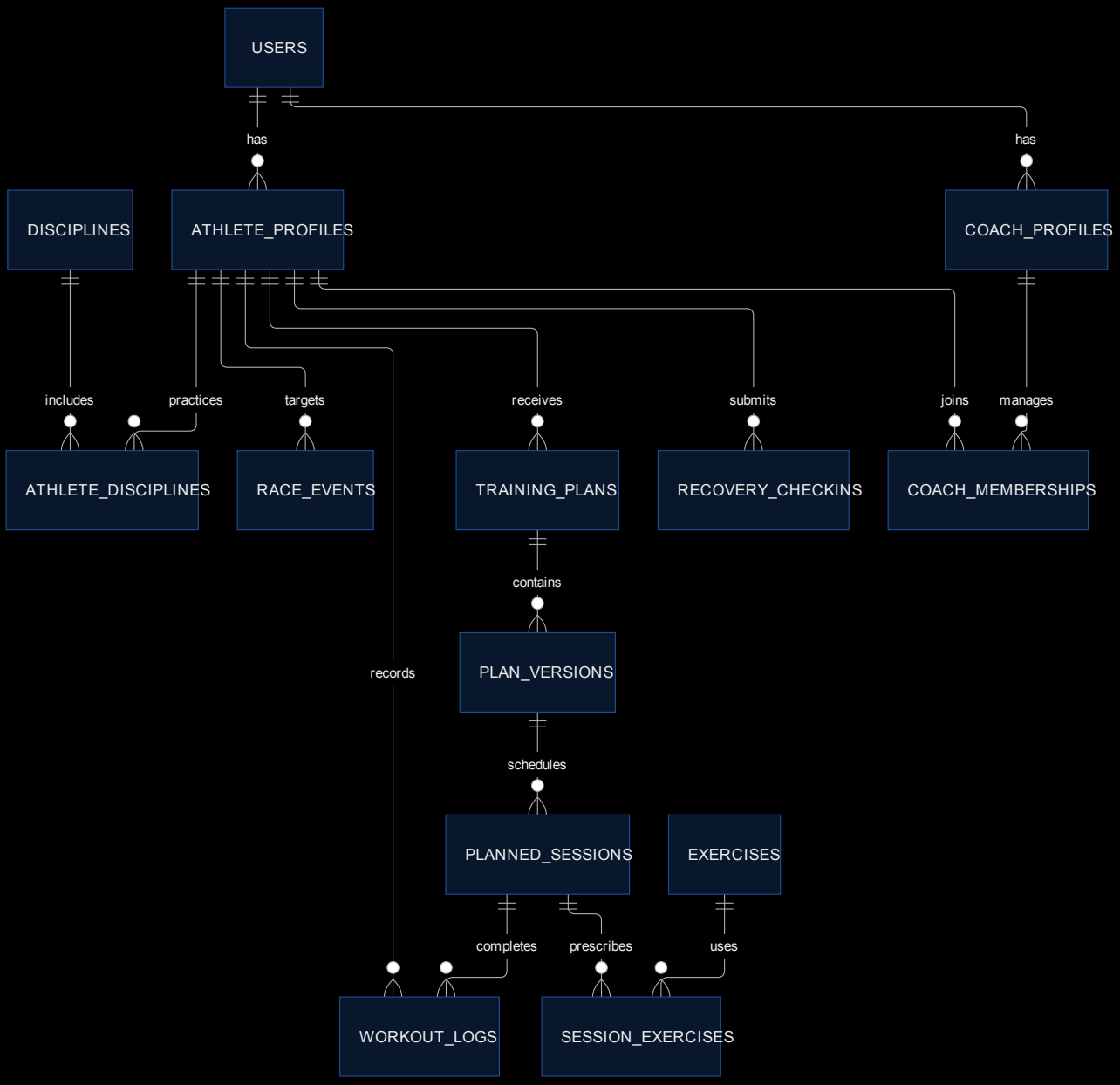
```
gravity-coach/
├── apps/
│   ├── web/                # Athlete and coach web experiences
│   ├── mobile/             # iOS and Android
│   ├── api/                # Backend API
│   └── worker/             # Background jobs
├── packages/
│   ├── domain/             # Core training and coaching rules
│   ├── schemas/            # Shared validation schemas
│   ├── api-client/         # Typed API client
│   ├── ui/                 # Reusable UI components
│   └── config/             # Shared linting and TypeScript config
├── database/
│   ├── migrations/
│   ├── seeds/
│   └── policies/
├── docs/
```

The exact tooling can evolve, but keep the coaching rules independent of the user interface. You should be able to test the training engine without launching the app.

4. Core data model

The database must represent athletes, coaching relationships, training methodology, prescribed sessions, and actual performance as separate concepts.

Entity relationships



Essential tables

Table	Important fields	Purpose
`users`	`id`, `email`, `created_at`	Identity and account

`athlete_profiles`	`user_id`, `birth_date` or age bracket, `timezone`, `preferences`	Athlete-specific settings
`coach_profiles`	`user_id`, `display_name`, `credentials`	Coach identity and professional information
`coach_memberships`	`coach_id`, `athlete_id`, `role`, `status`	Consent-based coach–athlete access
`disciplines`	`id`, `code`, `configuration`	DH, enduro, XC, and others
`athlete_disciplines`	`athlete_id`, `discipline_id`, `level`, `priority`	Multiple disciplines per athlete
`athlete_constraints`	`athlete_id`, `constraint_type`, `details`, `active`	Equipment, availability, and training limitations
`race_events`	`athlete_id`, `discipline_id`, `event_date`, `priority`	Race calendar
`exercises`	`id`, `movement_pattern`, `equipment`, `difficulty`, `instructions`	Exercise library
`training_templates`	`id`, `discipline_id`, `phase`, `version`	Reusable coaching methodology
`training_plans`	`id`, `athlete_id`, `status`, `created_by`	Overall training plan
`plan_versions`	`id`, `plan_id`, `version`, `methodology_version`, `snapshot`	Immutable record of prescriptions
`planned_sessions`	`id`, `plan_version_id`, `scheduled_at`, `session_type`, `status`	Calendar and daily sessions
`session_exercises`	`session_id`, `exercise_id`, `sets`, `reps`, `load_target`, `rest_seconds`	Prescribed workout details
`workout_logs`	`athlete_id`, `session_id`, `completed_at`, `RPE`, `duration`	Actual training performance
`recovery_checkins`	`athlete_id`, `date`, `sleep`, `energy`, `soreness`	Recovery feedback
`audit_events`	`actor_id`, `action`, `entity_id`, `timestamp`	Accountability and change history

This is the logical starting point, not a mandate to create every table on day one.

Four data-model decisions that matter

1. Separate the user from the athlete profile. A user may be both an athlete and a coach. A single account should support both roles without duplicating identity records.

2. Separate planned work from completed work. The planned session describes what the athlete was asked to do. The workout log describes what actually happened. Never overwrite the prescription with the performed workout.

3. Version training plans. When a coach changes a session or the engine adapts a plan, preserve the original prescription and record the reason for the change.

4. Support multiple disciplines without duplicate profiles. An athlete can race DH and enduro while sharing their equipment, training history, recovery information, and availability.

For flexible data such as questionnaire answers, JSONB can be useful. Keep frequently queried fields, relationships, permissions, and metrics in typed relational columns.

Multi-tenancy and permissions

At launch, the most important permissions are:

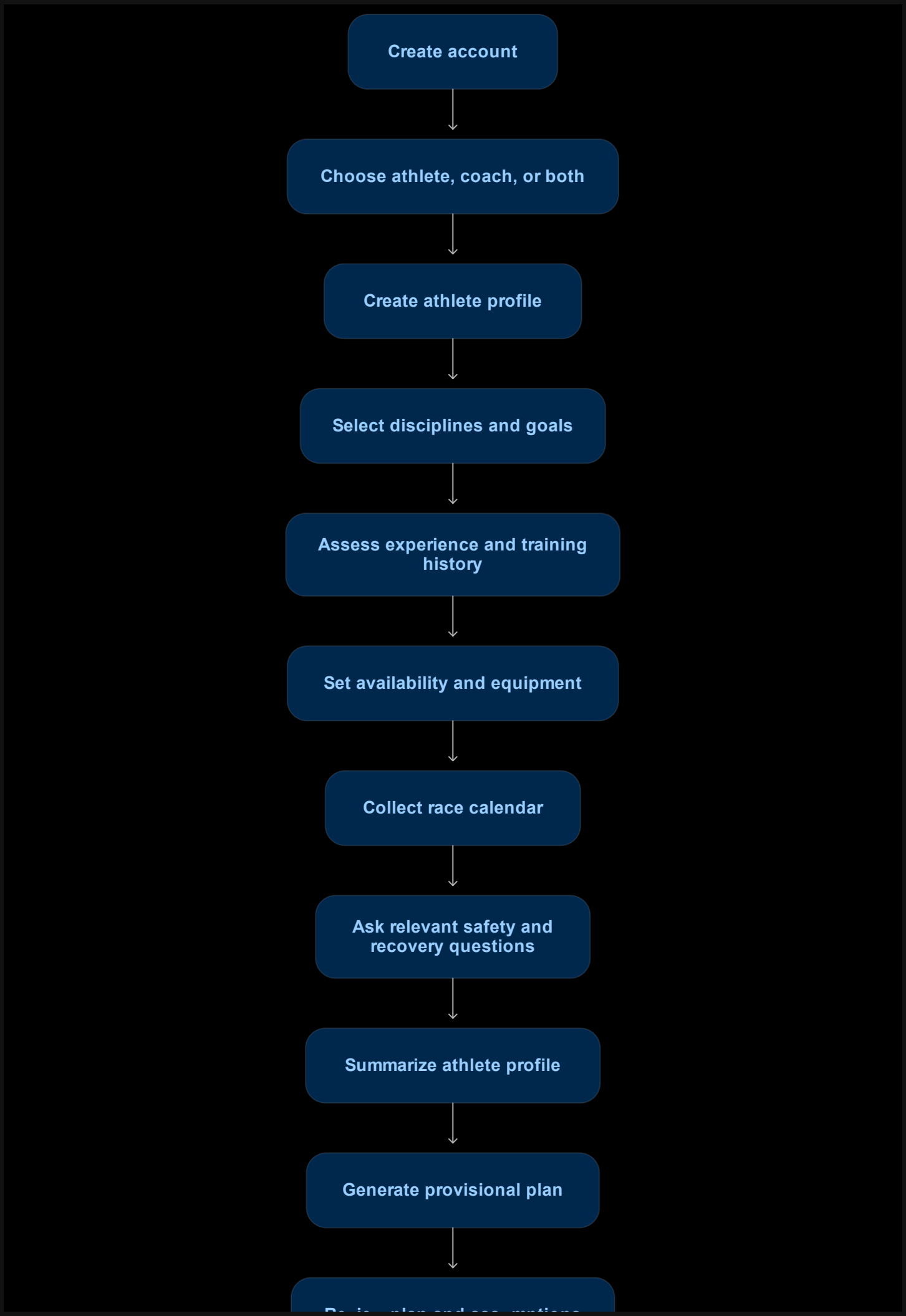
- Athletes can read and update their own permitted profile fields and log their own workouts.
- Coaches can access only athletes who have explicitly accepted an active coaching relationship.
- Coaches can prescribe or edit plans only within their authorized permissions.
- Team managers can administer their team without automatically gaining access to every private athlete field.
- Administrators can maintain platform content through separately controlled privileges.

Use database row-level security where appropriate, backend authorization on every protected operation, and automated cross-account access tests. Never trust a client-supplied athlete ID as proof of authorization.

Store dates and timestamps consistently, retain the athlete's timezone for scheduling, and make all sensitive data collection and retention explicit.

5. Athlete onboarding flow

Design onboarding as a guided sequence with branching questions. It should feel short for a beginner but provide enough depth for an advanced racer.


Create account
Choose athlete, coach, or both
Create athlete profile
Select disciplines and goals
Assess experience and training history
Set availability and equipment
Collect race calendar
Ask relevant safety and recovery questions
Summarize athlete profile
Generate provisional plan

Onboarding screens in detail

1. **Account and role**

   Ask whether the person is an athlete, a coach, or both. Coaches can create their coaching workspace immediately while completing an athlete profile only if they also train.

2. **Discipline and goals**

   Allow multiple selections and identify a primary discipline. Ask what success looks like: completing a first race, improving consistency, increasing strength, or targeting competitive performance.

3. **Experience assessment**

   Separate riding ability, racing experience, gym experience, and endurance conditioning. Ask about recent training, not just lifetime achievements.

4. **Resources and availability**

   Ask which days and times are available, typical session duration, equipment, trail access, and other regular commitments. Let athletes indicate unavailable days.

5. **Race and readiness assessment**

   Collect target dates, priority events, current fatigue, relevant limitations, and whether the athlete is returning from an injury. Ask only what is needed to determine safe next steps.

6. **Plan preview**

   Explain the proposed training phases, weekly time commitment, main priorities, and assumptions. Let the athlete correct their information before activation.

Dynamic questions instead of one enormous form

For example:

- If the athlete has no gym access, ask about available home equipment and alternatives.
- If the athlete has no race date, offer a general development plan rather than inventing a race deadline.
- If the athlete selects multiple disciplines, ask which is the primary focus and whether they have overlapping race calendars.
- If the athlete reports an active injury or concerning symptoms, do not automatically prescribe a normal high-intensity program.

- If a coach has invited the athlete, allow approved profile information to be shared through an explicit consent flow.

Keep onboarding resumable. Save progress between screens, show why sensitive questions are asked, and allow users to skip nonessential questions.

6. The personalized training engine

This is the core intellectual property of the platform.

I'd implement the engine as a **deterministic planning pipeline with explicit constraints**, not as a single prompt sent to an LLM.

Stage A — Build the athlete model

Normalize onboarding answers into a structured representation.

TypeScript

```
type AthleteModel = {
  athleteId: string;

  disciplines: Array<{
    code: "dh" | "enduro" | "xc" | "freeride";
    level: "beginner" | "intermediate" | "advanced" | "elite";
    priority: number;
  }>;

  training: {
    strengthExperience: "novice" | "intermediate" | "advanced";
    availableDays: number[];
    minutesPerSession: number;
    equipmentIds: string[];
    ridingAccess: string[];
  };

  goal: {
```

This is a simplified domain type. Production code should validate all inputs, use a defined schema, and represent dates and measurements consistently.

Don't infer precise strength levels, fitness scores, or injury severity from broad questionnaire answers. Track what is known, what is self-reported, and what is unknown.

Stage B — Determine the training phase

The phase should depend on race dates, current preparation, training history, and the athlete's ability to tolerate the intended workload.

A useful phase model is:

Phase	Primary focus
General preparation	Movement quality, basic strength, aerobic foundation
Strength development	Progressive force production and physical capacity
Power and specificity	Explosive qualities, discipline-specific conditioning
Competition preparation	Race-specific performance and skill practice
Taper and competition	Reduce accumulated fatigue while preserving readiness
Transition and recovery	Recover, review the previous cycle, and rebuild gradually

Not every athlete needs a full 12-week sequence. If someone starts six weeks before a race, the engine should not blindly compress all twelve weeks into six. It should prioritize appropriate adaptations, account for existing fitness, and avoid abrupt workload spikes.

For athletes with multiple disciplines, use one shared calendar and coordinate competing priorities rather than generating independent plans that overload the same week.

Stage C — Allocate the weekly training budget

The engine first determines the available training slots and the demands of each session.

For example, if an athlete can train four days per week for an hour per session, the engine cannot schedule six hours of gym work plus three hours of riding.

A simplified allocation model:

Plain text

```
1. Reserve required recovery and unavailable days.
2. Place important technical riding sessions.
3. Place strength sessions with adequate recovery.
4. Add conditioning where it supports the current phase.
5. Fit warm-ups, mobility, and cooldowns into session duration.
6. Check total workload and conflicts.
7. If the plan does not fit, reduce optional volume or revise priorities.
```

The priority order is configurable. A coach preparing an elite DH racer for an upcoming event may choose a different distribution than a beginner building general fitness.

Stage D — Select and prescribe exercises

Each exercise needs metadata and progression rules.

TypeScript

```typescript
type ExerciseDefinition = {
  id: string;
  name: string;
  movementPattern: string;
  requiredEquipment: string[];
  difficulty: "beginner" | "intermediate" | "advanced";
  trainingEffects: string[];
  contraindicationFlags: string[];
  regressions: string[];
  progressions: string[];
  defaultPrescription: {
    sets: number;
    reps?: number;
    durationSeconds?: number;
    restSeconds: number;
  };
};
```

The engine filters exercises based on equipment, skill, constraints, and intended training effect, then builds a coherent session.

It should not merely substitute any exercise that targets the same muscle. Substitutions must consider movement pattern, technical complexity, loading, and the reason the original exercise was prescribed.

Stage E — Validate the plan

Before saving a generated plan, run checks for:

- Session duration and scheduling feasibility.
- Appropriate training frequency and recovery.
- Exercise prerequisites and equipment.
- Abrupt changes in intensity or volume.
- Conflicting high-intensity sessions.
- Phase-appropriate training.
- Race-date and taper consistency.

- Known limitations and safety rules.
- Missing or contradictory profile data.

If a plan fails validation, revise it or ask the athlete for more information. Don't silently bypass constraints.

Stage F — Explain every prescription

Every session should include a short explanation of why it exists, what the athlete should focus on, and how success is measured.

For example:

Lower-body strength
Example session · Intermediate athlete

55 min



Goblet squat

3 sets × 8 reps

Rest 90–120 seconds. Use a controlled range of motion and finish each set with good technique.



Romanian deadlift

3 sets × 8 reps

Rest 2 minutes. Maintain a controlled hip hinge and a comfortable range of motion.



Dead bug

3 sets × 8 per side

Rest as needed. Maintain trunk control throughout each repetition.

Why this session?

Develop foundational lower-body strength and trunk control while keeping fatigue manageable for the remainder of the training week.

Illustrative workout only; actual prescriptions must be generated from the athlete's profile and training phase.

Stage G — Adapt the plan from actual training

After each session, collect:

- Whether it was completed, partially completed, or skipped.
- Actual duration and perceived exertion.

- Exercise loads and repetitions where relevant.
- Athlete-reported fatigue, soreness, and energy.
- Optional performance results, such as repeatable course times.

Then determine whether to maintain, progress, reduce, or replace future sessions.

For example, progression should require sufficient evidence that the athlete completed the work with appropriate technique and tolerable effort. A single easy session should not trigger an automatic jump in training load.

The engine should also distinguish *missed sessions* from *poor recovery*. A missed workout caused by scheduling is not the same as an athlete being too fatigued to train.

Where AI fits

Use AI for tasks where language and contextual reasoning add value:

- Explaining why a workout was prescribed.
- Answering questions using your approved coaching knowledge base.
- Summarizing a week's training for a coach.
- Proposing alternative sessions for review.
- Helping coaches draft notes and feedback.

Keep plan generation, authorization, scheduling constraints, load limits, and safety validation in controlled application code. AI suggestions must be validated before they become prescriptions.

7. API design

Use a versioned REST API initially. It is straightforward to document, secure, and consume from both Next.js and Expo.

Base path: `/api/v1`

Resource	Example endpoints	Responsibility
Authentication	Managed auth endpoints	Sign-in, sessions, recovery
Athlete profiles	`GET/PATCH /athletes/me`	Profile and preferences
Disciplines	`GET /disciplines`	Available discipline definitions
Onboarding	`GET/PATCH /athletes/me/onboarding`	Save assessment progress
Race calendar	`GET/POST /athletes/me/events`	Events and priorities
Training plans	`GET /athletes/me/plans`	Active and historical plans

Plan generation	`POST /athletes/me/plans/generate`	Request a new plan
Sessions	`GET /sessions/:id`	Prescriptions and instructions
Workout logs	`POST /sessions/:id/logs`	Record completed work
Recovery	`POST /athletes/me/check-ins`	Readiness and fatigue feedback
Coach roster	`GET /coach/athletes`	Authorized athlete management
Coach assignments	`POST /coach/athletes/:id/plans`	Assign or modify a plan
Exercise library	`GET /exercises`	Search approved exercises
AI coaching	`POST /coach-assistant/messages`	Validated coaching assistance

For expensive operations such as generating a full training plan, return a job identifier and let the client monitor progress. Use idempotency keys for operations that must not create duplicate plans or logs when a mobile connection retries.

Every endpoint needs authorization tests, request validation, rate limits where appropriate, and predictable error responses.

8. Coach workspace

Coaches need more than access to the athlete's individual app. Build a dedicated workspace around their actual workflow.

Coach dashboard concept

Athletes
Roster and status

Plans
Review and assignment

Progress
Performance trends

Reviews
Feedback and flags

Essential coach workflows

- Invite an athlete and establish an authorized relationship.
- Review the athlete's profile and upcoming races.
- Generate a proposed plan using the shared engine.
- Review, edit, approve, and assign the plan.

- See completed workouts and athlete-reported fatigue.
- Leave feedback and make documented changes.

An important product decision: **the coach should be able to override the engine without fighting it**. When a coach changes a prescribed workout, preserve the original recommendation, the final prescription, the author, and the reason for the change.

The system can later learn which recommendations coaches frequently modify, helping you improve your methodology and defaults.

9. Concrete implementation roadmap

I'd plan the initial production build as a sequence of approximately 16–20 weeks, assuming a small experienced team. A solo developer or a team new to mobile development should expect a longer schedule. This is a planning estimate, not a delivery guarantee.

Weeks 1–2

Foundation

Milestone 1 — Product specification

- Confirm supported disciplines and athlete categories.
- Document the coaching methodology and safety boundaries.
- Create user journeys and clickable interface wireframes.
- Define the initial database schema and API contracts.
- Agree on pilot success metrics.

Exit criterion: an approved MVP specification and a testable domain model.

Weeks 3–5

Platform

Milestone 2 — Core infrastructure

- Establish the monorepo, development environments, and CI/CD.
- Implement authentication, profiles, permissions, and migrations.
- Build athlete onboarding and discipline configuration.
- Create coach–athlete invitations and authorization checks.
- Add automated tests for cross-account access.

Exit criterion: users can sign in, complete profiles, and establish secure coaching relationships.

Weeks 6–9

Coaching

Milestone 3 — Training engine

- Build and seed the exercise library.
- Implement training phases and session templates.
- Generate weekly schedules using deterministic rules.
- Add plan validation, versioning, and coach review.
- Test contrasting athlete profiles and edge cases.

Exit criterion: valid plans can be generated, explained, reviewed, and stored reproducibly.

Weeks 10–12

Experience

Milestone 4 — Athlete and coach apps

- Build the training calendar and workout detail screens.
- Add workout logging and recovery check-ins.
- Implement the coach roster and plan approval workflow.
- Add plan edits, change history, and useful progress views.
- Ensure web and mobile share consistent domain behavior.

Exit criterion: an athlete can follow a plan and a coach can manage it end to end.

Weeks 13–16

Adaptation

Milestone 5 — Feedback and AI

- Implement rules-based workload adjustments.
- Add missed-session rescheduling and basic notifications.
- Integrate the AI coaching gateway with approved content.
- Validate AI suggestions against coaching rules.
- Add monitoring, audit events, and operational alerts.

Exit criterion: the platform can adjust future sessions while preserving safety and plan history.

Weeks 17–20

Validation

Milestone 6 — Pilot and launch readiness

- Test iOS, Android, desktop, and mobile web.
- Perform security, backup, recovery, and load tests.
- Recruit pilot coaches and athletes.
- Review the usefulness and appropriateness of generated plans.
- Fix onboarding friction, confusing prescriptions, and performance issues.

Exit criterion: pilot users can complete the core workflow reliably and the team has clear evidence for the next iteration.

Suggested team

For this scope, a lean experienced team might include:

- One product lead with access to experienced coaches.
- One strength and conditioning / MTB coaching expert responsible for methodology.
- One or two full-stack engineers.
- One mobile-capable engineer, potentially overlapping with full-stack responsibilities.
- Part-time product design and quality assurance.

- Security and privacy review before launch.

The most easily underestimated work is not the calendar UI or the AI chat. It is developing and validating a coherent coaching methodology across disciplines and athlete levels.

10. Quality, safety, and success metrics

The platform should measure whether it helps athletes train consistently and whether its prescriptions are appropriate—not simply whether it generates plans quickly.

Area	Initial metric
Onboarding	Completion rate and abandonment by step
Plan generation	Valid plans produced without manual correction
Personalization	Proportion of sessions fitting stated equipment and schedule
Athlete engagement	Weekly active athletes and session completion
Coach workflow	Time needed to review and assign a plan
Adaptation	Frequency and reasons for plan changes
Safety	Invalid prescriptions, constraint violations, and unresolved flags
Reliability	API errors, failed background jobs, and recovery success

Set explicit acceptance criteria before the pilot. For example, all generated plans should pass automated scheduling, equipment, and authorization checks. Coaches should review the quality of prescriptions across representative profiles, and any serious safety issue should block release until resolved.

Avoid treating workout completion or lower perceived exertion as proof that an athlete is getting faster. Track objective performance where it is available, and account for course conditions, weather, and equipment when comparing race or riding times.

11. Decisions to lock before implementation

A few decisions will prevent costly rework.

- **Architecture**

  Shared TypeScript domain, web and mobile clients, versioned API, relational database, and a modular training engine.

- **Coaching model**

  Deterministic prescriptions and validation first, with AI as a constrained assistant.

- **Data ownership**

  Athlete-controlled profiles, explicit coach access, immutable plan versions, and separate workout logs.

- **Training methodology**

  Decide which disciplines and levels will be fully supported at launch, and identify the qualified coaches who will validate the initial templates.

- **Commercial model**

  Decide whether individual subscriptions, coach subscriptions, team licensing, or a combination will determine account entitlements.

My recommendation for the first engineering sprint

Start with three concrete deliverables:

1. A versioned database schema and authorization model.
2. An exercise library and a small, reviewed set of training templates covering at least three contrasting disciplines.
3. A working onboarding-to-plan-generation flow that produces a reviewable weekly plan from a structured athlete profile.

That gives you the core of the product early. You can then build the calendar, logging, coach dashboard, and mobile experience on top of a functioning coaching engine instead of designing the whole interface around assumptions.

The next artifact I would create is a complete technical specification: database migrations, API contracts, domain types, plan-generation interfaces, authorization rules, and test cases. That specification would be the handoff document for implementation and the foundation for the first working release.